# RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Department of Artificial Intelligence | V Semester B.Tech AIDS

**AD23532 – Principles of Data Science**

# Ex2: Data Discovery and Preparation

**Name:** Praveen Kumar S **Roll No:** 241801212 **Batch:** 2024-2028

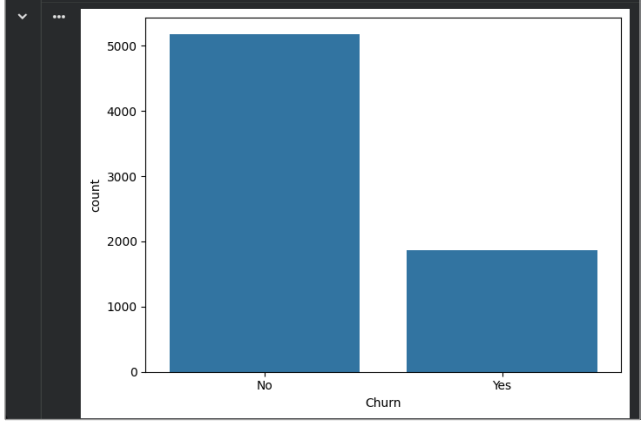
## Task 1 – Reuse Ex:1 Steps 1–6, 10

Ex:1 Steps 1–6 and 10 (library import, data loading, exploration, cleaning, encoding, scaling and correlation heatmap) are reused; refer to the Ex1 record.

## Task 2 – Analyse the distribution of Customer Churn, Gender, Contract Type and Payment Method

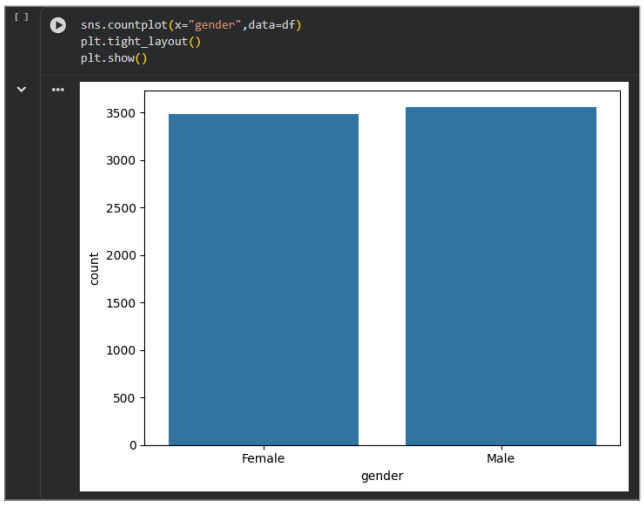
*Customer Churn*

```
sns.countplot(x="Churn",data=df)
plt.tight_layout()
plt.show()
```



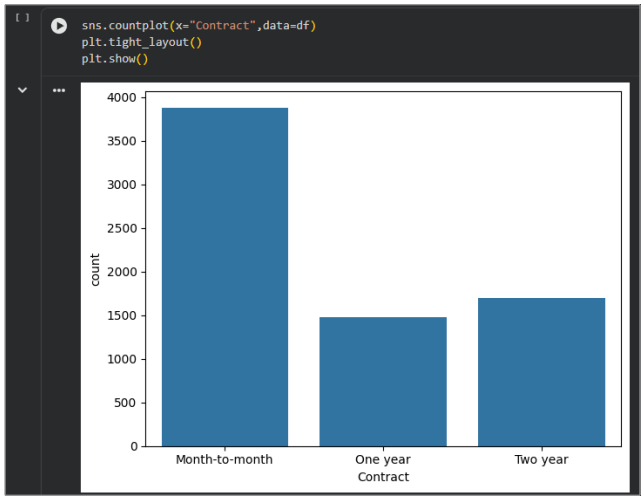

*Gender*

```
sns.countplot(x="gender",data=df)
plt.tight_layout()
plt.show()
```



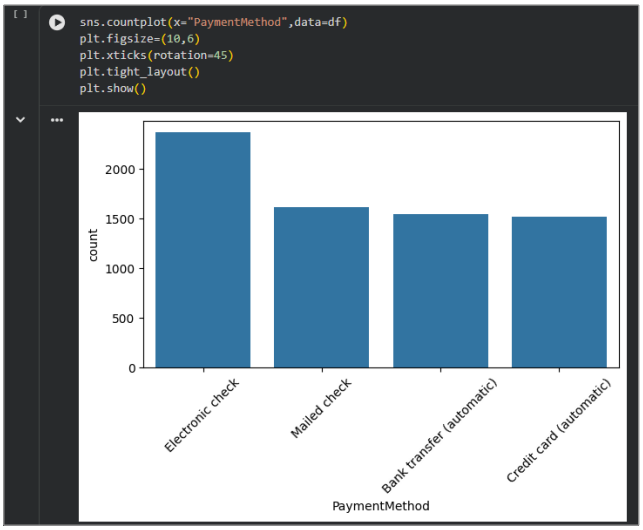

*Contract Type*

```
sns.countplot(x="Contract",data=df)
plt.tight_layout()
plt.show()
```




*Payment Method*

```
sns.countplot(x="PaymentMethod",data=df)
plt.figsize=(10,6)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



## Task 3 – Histograms and boxplots of MonthlyCharges, TotalCharges and Tenure

*Code*

```
# Visualize 'MonthlyCharges'
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1) # 1 row, 2 columns, 1st plot
plt.hist(df['MonthlyCharges'], bins=30, edgecolor='black')
plt.title('Distribution of Monthly Charges')
plt.xlabel('Monthly Charges')
plt.ylabel('Frequency')

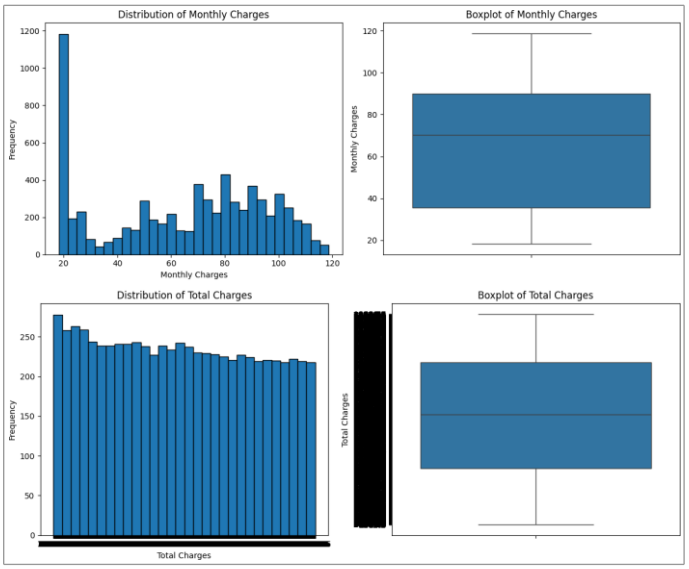
plt.subplot(1, 2, 2) # 1 row, 2 columns, 2nd plot
sns.boxplot(y=df['MonthlyCharges'])
plt.title('Boxplot of Monthly Charges')
plt.ylabel('Monthly Charges')
plt.tight_layout()
plt.show()

# Visualize 'TotalCharges'
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.hist(df['TotalCharges'], bins=30, edgecolor='black')
plt.title('Distribution of Total Charges')
plt.xlabel('Total Charges')
plt.ylabel('Frequency')

plt.subplot(1, 2, 2)
sns.boxplot(y=df['TotalCharges'])
plt.title('Boxplot of Total Charges')
plt.ylabel('Total Charges')
plt.tight_layout()
plt.show()

# Visualize 'tenure'
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.hist(df['tenure'], bins=30, edgecolor='black')
plt.title('Distribution of Tenure')
plt.xlabel('Tenure (months)')
plt.ylabel('Frequency')
```

*Output (MonthlyCharges, TotalCharges)*




## Task 4 – Detect outliers using the Interquartile Range (IQR) method

*Code*

```
4. Detect outliers in numerical features using the Interquartile Range (IQR) method.

numerical_cols = df.select_dtypes(include=np.number).columns

outlier_summary = {}

for col in numerical_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
    num_outliers = len(outliers)

    outlier_summary[col] = {
        'Q1': Q1,
        'Q3': Q3,
        'IQR': IQR,
        'Lower Bound': lower_bound,
        'Upper Bound': upper_bound,
        'Number of Outliers': num_outliers,
        'Percentage of Outliers': (num_outliers / len(df)) * 100
    }

    print(f"\n--- Outlier Analysis for {col} ---")
    print(f"Q1: {Q1:.2f}")
    print(f"Q3: {Q3:.2f}")
    print(f"IQR: {IQR:.2f}")
    print(f"Lower Bound: {lower_bound:.2f}")
    print(f"Upper Bound: {upper_bound:.2f}")
```

*Output – outlier summary*

```
outlier_df = pd.DataFrame(outlier_summary).T
display(outlier_df)

--- Outlier Analysis for SeniorCitizen ---
Q1: 0.00
Q3: 0.00
IQR: 0.00
Lower Bound: 0.00
Upper Bound: 0.00
Number of Outliers: 1142
Percentage of Outliers: 16.21%

--- Outlier Analysis for tenure ---
Q1: 9.00
Q3: 55.00
IQR: 46.00
Lower Bound: -60.00
Upper Bound: 124.00
Number of Outliers: 0
Percentage of Outliers: 0.00%

--- Outlier Analysis for MonthlyCharges ---
Q1: 35.50
Q3: 89.85
IQR: 54.35
Lower Bound: -46.02
Upper Bound: 171.38
Number of Outliers: 0
Percentage of Outliers: 0.00%
```

| | Q1 | Q3 | IQR | Lower Bound | Upper Bound | Number of Outliers | Percentage of Outliers |
|---|---|---|---|---|---|---|---|
| SeniorCitizen | 0.0 | 0.00 | 0.00 | 0.000 | 0.000 | 1142.0 | 16.214681 |
| tenure | 9.0 | 55.00 | 46.00 | -60.000 | 124.000 | 0.0 | 0.000000 |
| MonthlyCharges | 35.5 | 89.85 | 54.35 | -46.025 | 171.375 | 0.0 | 0.000000 |

## Task 6 – Analytical questions

***Q1. Which contract type has the highest customer churn?*** *Month-to-month (churn rate ≈ 42.7%).*

Which contract type has the highest customer churn?

```
churn_by_contract = df.groupby(['Contract', 'Churn']).size().unstack(fill_value=0)
churn_by_contract['Churn_Rate'] = churn_by_contract['Yes'] / (churn_by_contract['No'] + churn_by_contract['Yes'])
display(churn_by_contract)

highest_churn_contract = churn_by_contract['Churn_Rate'].idxmax()
print(f"The contract type with the highest churn rate is: {highest_churn_contract}")
```

| Contract | No | Yes | Churn_Rate |
|---|---|---|---|
| Month-to-month | 2220 | 1655 | 0.427097 |
| One year | 1307 | 166 | 0.112695 |
| Two year | 1647 | 48 | 0.028319 |

```
The contract type with the highest churn rate is: Month-to-month
```

***Q2. Which payment method is most frequently used by customers who churn?*** *Electronic check (1071 churned customers).*

Which payment method is most frequently used by customers who churn?

```
churned_customers = df[df['Churn'] == 'Yes']
most_frequent_payment_method = churned_customers['PaymentMethod'].value_counts().idxmax()
print(f"The payment method most frequently used by customers who churn is: {most_frequent_payment_method}")
display(churned_customers['PaymentMethod'].value_counts())
```

```
The payment method most frequently used by customers who churn is: Electronic check
```

| PaymentMethod | count |
|---|---|
| Electronic check | 1071 |
| Mailed check | 308 |
| Bank transfer (automatic) | 258 |
| Credit card (automatic) | 232 |

dtype: int64

***Q3. Do customers with shorter tenure tend to churn more often?*** *Yes – average tenure is 17.98 months for churned vs 37.57 months for non-churned customers.*

Do customers with shorter tenure tend to churn more often?

```
mean_tenure_churn_yes = df[df['Churn'] == 'Yes']['tenure'].mean()
mean_tenure_churn_no = df[df['Churn'] == 'No']['tenure'].mean()

print(f"Average tenure for churned customers: {mean_tenure_churn_yes:.2f} months")
print(f"Average tenure for non-churned customers: {mean_tenure_churn_no:.2f} months")

if mean_tenure_churn_yes < mean_tenure_churn_no:
    print("Yes, customers with shorter tenure tend to churn more often.")
else:
    print("No, customers with shorter tenure do not tend to churn more often (or the difference is negligible).")
```

```
Average tenure for churned customers: 17.98 months
Average tenure for non-churned customers: 37.57 months
Yes, customers with shorter tenure tend to churn more often.
```

## Task 7 – Split the cleaned dataset into Training (80%) and Testing (20%)

*Train/test split using* `train_test_split(X, y, test_size=0.2, random_state=42)`

```
from sklearn.model_selection import train_test_split

# Define features (X) and target (y)
X = df_encoded.drop(['Churn_Yes', 'customerID'], axis=1) # Drop 'customerID' as it's not a feature
y = df_encoded['Churn_Yes']

# Split the dataset into training (80%) and testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Training set shape (X_train, y_train):")
```

## Task 8 – Save the final processed dataset

*Processed dataset saved as* `CustomerChurn_Preprocessed.csv`

```
# Save the final processed dataset as a new CSV file
df_encoded.to_csv('CustomerChurn_Preprocessed.csv', index=False)
print("Processed dataset saved to 'CustomerChurn_Preprocessed.csv'")
```

```
Processed dataset saved to 'CustomerChurn_Preprocessed.csv'
```